

Early-Stopping Thresholds for ES-HyperNEAT: A Data-Driven Approach from Fitness Dynamics

Romain Claret^{*†}, Arthur Gygax^{*‡}, Michael O'Neill[§], Paul Cotofrei^{*†}, and Pascal Felber^{*‡}

^{*}University of Neuchâtel, Switzerland

[†]Information Management Institute

[‡]Computer Science Department

[§]Natural Computing Research & Applications Group, University College Dublin, Ireland

Abstract—Most hyperparameter configurations for Evolvable-Substrate HyperNEAT (ES-HyperNEAT) produce networks that stagnate at random-guessing performance, wasting computational resources. We frame early stopping as binary classification on early fitness trajectories: for each trial, we compute the cumulative median of best-per-generation fitness and test it against a threshold derived by maximizing the F_1 score on an initial 90-trial dataset. The resulting rule (generation $G^*=3$, threshold $T^*=0.140$) achieves $F_1=0.872$ on 180 independent validation trials, retaining over 90% of successful trials while cutting computational cost by 41.6%. Compared to Hyperband, our domain-specific rule is 64% more efficient with higher mean fitness, though Hyperband occasionally discovers higher peak solutions. On a converged search population the rule becomes too aggressive (recall 31.1%), motivating adaptive thresholds. The specific thresholds are ES-HyperNEAT-specific, but the methodology, deriving stopping criteria from fitness dynamics classification, is applicable to other evolutionary algorithms with stagnation-prone hyperparameter spaces.

Index Terms—Data-driven pruning, early stopping, ES-HyperNEAT, hyperparameter optimization, neuroevolution

I. INTRODUCTION

Neuroevolutionary algorithms, such as the NeuroEvolution of Augmenting Topologies (NEAT) [1] and its indirect encoding extension ES-HyperNEAT [2], can evolve neural network architectures without backpropagation. Their performance, however, depends heavily on hyperparameter choices, and the irregular search space of ES-HyperNEAT makes selecting good configurations difficult.

The Tree-structured Parzen Estimator (TPE) [3] offers a structured way to search such spaces. Previous work [4] showed that TPE outperforms random search on ES-HyperNEAT classification benchmarks including MNIST, reaching higher accuracy with fewer evaluations by learning the structure of the search space.

One problem persists regardless of the search method: most configurations produce trials that show no learning and stagnate near random-guessing performance. These dead-end runs waste time without informing the search.

We ask three questions. (1) Can early fitness dynamics reliably predict whether an ES-HyperNEAT trial will succeed or fail? (2) How early in the evolutionary run can this

prediction be made? (3) Does a domain-specific stopping rule derived from these dynamics outperform general-purpose pruning schedules?

We propose an early-stopping strategy that kills unpromising runs before they finish. We observe that failing trials show a characteristic flat fitness trajectory within the first few generations, and we frame the pruning decision as binary classification: given the median best fitness up to a cutoff generation, is this trial going to succeed or not? The resulting rule is simple and interpretable, and it keeps most successful trials while cutting wasted computation.

We validate the rule on held-out trials and show that it generalizes. Integrating early stopping with TPE-driven search speeds up exploration without degrading solution quality.

II. RELATED WORK

The optimization of neural networks via evolutionary algorithms, a field known as neuroevolution, offers an alternative to gradient-based methods. The NeuroEvolution of Augmenting Topologies (NEAT) algorithm [1] evolves both the weights and the structure of networks, starting from minimal configurations. This approach of complexification, where topologies grow to match the problem's complexity, helps to minimize the dimensionality of the search space. ES-HyperNEAT [2] extends this paradigm with an indirect encoding that generates regular network patterns on an evolvable substrate, allowing it to handle high-dimensional problems. The performance of these algorithms, however, depends on many interacting hyperparameters, making manual tuning impractical. We use ES-HyperNEAT with TPE-based hyperparameter optimization.

For hyperparameter optimization (HPO), Bayesian Optimization (BO) is a sample-efficient strategy [5]. Unlike grid or random search [6], BO builds a probabilistic surrogate model of the objective function and uses it to select configurations likely to yield improvements. The Tree Structured Parzen Estimator (TPE) [3], a popular BO algorithm, models separate probability distributions $\ell(x) = p(x \mid y < y^*)$ for well-performing configurations and $g(x) = p(x \mid y \geq y^*)$ for the remainder, then maximizes the ratio $\ell(x)/g(x)$ to propose new candidates. Unlike GP-based BO, TPE handles categorical and conditional variables natively and scales to large trial budgets, making it well suited to the mixed hyperparameter space of

Romain Claret and Arthur Gygax contributed equally to this work.
Corresponding author: Romain Claret (romain.claret@unine.ch).

ES-HyperNEAT. TPE outperforms random search and other BO methods on many ML tasks. These methods improve search efficiency but do not reduce the cost of individual evaluations.

The high cost of evaluating many configurations has led to the development of early-stopping, or pruning, strategies. These methods aim to terminate unpromising trials early, reallocating computational resources to more promising candidates. Throughout this paper, “pruning” refers to early termination of hyperparameter trials, not network pruning (removing connections from neural networks), which is a separate technique in neuroevolution. Prominent examples include the Successive Halving Algorithm (SHA) [7] and Hyperband [8], which periodically evaluate a portfolio of configurations and discard a fixed proportion of the worst performers. Other approaches attempt to predict the final performance of a trial by extrapolating its initial learning curve [9], allowing for early termination if the predicted outcome is not competitive. Frameworks like Optuna have integrated such mechanisms, for instance, by stopping trials that perform worse than the median of their peers at an intermediate step [10].

These general-purpose pruning methods work well but rely on fixed-elimination schedules or complex predictive models. We take a different approach: instead of predefined schedules or learning-curve extrapolation, we derive a pruning rule directly from the fitness dynamics of ES-HyperNEAT. Framing the pruning decision as binary classification, we obtain an interpretable, empirically grounded rule that separates successful from unsuccessful trials at the earliest possible generation.

ES-HyperNEAT [2] evolves both weights and topology using an indirect encoding: a Compositional Pattern-Producing Network (CPPN) [11] generates connectivity patterns, and an adaptive substrate mechanism sets topology through quadtree variance analysis. Unlike fixed-topology networks where a few learning-rate and regularization parameters dominate, ES-HyperNEAT exposes 16 interacting hyperparameters spanning NEAT mutation rates, population size, substrate depth, and variance thresholds for quadtree subdivision. Most configurations in this space produce networks that never learn, making ES-HyperNEAT a challenging and representative testbed for early-stopping research.

III. EXPERIMENTAL SETUP

Given the stagnation-prone nature of the ES-HyperNEAT hyperparameter space described above, we introduce an early-stopping strategy to terminate unpromising trials before completion. The pruning decision is formulated as a threshold test on the cumulative median best fitness (Equation 1). The methodology has two phases: Phase 1 (*Derivation*) establishes the pruning criterion via classification on initial trials, while Phase 2 (*Validation*) tests the rule on new, independent trials.

The experiments were implemented using Python and the PUREPLES framework [12], which is based on NEAT-Python [13], and were conducted exclusively on CPUs across 14 machines with varying CPU capabilities and RAM sizes using the Optuna framework [10]. This hardware variability pre-

cluded direct wall-clock time comparisons; all efficiency metrics are therefore reported in generations rather than elapsed time. All evolutionary trials were run for a maximum of 17 generations, based on prior analysis [4] showing only 2.7% performance loss versus generation 20, with lower variance, justifying the per-trial computational savings for large-scale experimentation. We use MNIST as a standard benchmark for studying ES-HyperNEAT search dynamics, following prior work [4]. The absolute classification accuracy is not the focus of this study.

A. Fitness Evaluation and Metrics

We used the same sixteen hyperparameters as in prior work [4], which provides the base search space specification. To enable broader exploration, the ranges were extended for several parameters: mutation probabilities (`conn_add/delete_prob`, `node_add/delete_prob`) from 3–4 discrete values to 0.1–0.9 in steps of 0.1; `pop_size` from {10, 20, 40, 60, 80, 100} to 30–130 in steps of 10; substrate depth parameters (`initial_depth` to 1–4, `max_depth` to 2–6); and threshold parameters (`variance_threshold`, `division_threshold`) to finer granularity. This extended search space contains over 3 billion potential configurations. For the hyperparameters not included in the search space, we used the configuration file from the XOR problem experiment in the NEAT-Python library and left all other parameters at the library’s default values.

The fitness of each genome was evaluated at each generation using a batch of 200 images randomly sampled from the MNIST training set, with images spread evenly across all 10 classes (20 per class). Each genome evaluation drew a fresh random batch: images were resampled independently for every genome at every generation, not fixed at trial initialization. This per-evaluation resampling prevents overfitting to any fixed subset. The class-balanced design and the use of cumulative median best fitness as the pruning metric together mitigate the resulting stochastic evaluation noise. A phenotype network’s prediction was determined by the index of the maximum value in its output vector. For each correct prediction, the genome’s fitness score was incremented by one, and the total score was averaged over the batch. This setup results in a baseline fitness of 0.1 for a network that makes random or constant predictions.

For the MNIST experiment, the substrate was initialized at the center of the input and output vectors. The objective value for TPE optimization was the best fitness achieved by the best-performing individual in the final generation.

While the primary pruning strategy relies on a single metric, additional metrics were recorded for detailed analysis. A key distinction is made between metrics that capture performance at a single generation versus cumulative metrics that track the historical trend of a trial. The most important metrics are defined below:

Last Generation Best Fitness: The fitness of the single best-performing individual in the trial’s final generation. This serves as the objective value for TPE optimization.

Best Fitness Over All Generations: The peak fitness achieved by any individual in any generation throughout the trial.

Median Best Fitness: (Core Pruning Metric) At generation G , this is the median of the set of best-per-generation fitness values recorded from generation 1 up to G . This cumulative statistic, chosen over alternative metrics for outlier resistance (see Section IV-F), forms the basis of our pruning rule.

Mean Best Fitness: The mean of the set of best-per-generation fitnesses, calculated identically to the median version.

B. Phase 1: Pruning Rule Derivation

To determine the optimal pruning generation G^* and fitness threshold T^* , the problem was framed as a binary classification task: predicting the final success of a trial based on its early fitness trajectory.

1) *Trial Labeling and Feature Selection:* An initial dataset was generated by sampling 30 distinct hyperparameter configurations using Optuna’s random sampler, with each configuration replicated 3 times ($N=90$ trials). This 30×3 design balances breadth and variance: 30 unique configurations provide diverse coverage of the large search space, while three stochastic repeats per configuration reduce the risk of labeling a setting based on a single lucky/unlucky evolutionary run and yield a more stable estimate of early fitness dynamics for threshold selection. Each trial was run to completion (17 generations) and labeled based on its final median best fitness:

- *Positive (Successful):* Trials with a final fitness in the 80th percentile or higher.
- *Negative (Unsuccessful):* All other trials.

The 80th percentile was selected to isolate the top-performing runs, in line with the Pareto principle, and to provide a stable, data-relative definition of success. In absolute terms, this cutoff corresponds to a final fitness of approximately 0.18 (about 18% MNIST accuracy, versus the 0.1 random-guessing baseline). This choice is also justified by empirical results presented in Section IV-A. The predictive feature for classification was the median best fitness calculated up to a given generation.

2) *Selecting the Pruning Generation G^* :* We identified G^* as the earliest generation where successful trials consistently showed learning. For each Positive-labeled trial, we identified the first generation where median best fitness exceeded the 0.1 baseline, then computed the ceiling of the mean of these generation numbers. This method roots the pruning point in the empirical learning dynamics of high-potential trials.

3) *Selecting the Pruning Threshold T^* :* At the selected pruning generation G^* , a threshold sweep was conducted to find the optimal fitness cutoff T^* . Every unique median best fitness value observed at generation G^* across the 90 trials was evaluated as a potential threshold. For each threshold,

a confusion matrix was generated and the F_1 score was calculated. The threshold T^* was selected as the one that maximized the F_1 score, achieving the best balance between precision and recall.

The resulting rule is simple, interpretable, and empirically grounded. Formally, let $\tilde{f}(t, g)$ denote the median of the best-per-generation fitness values for trial t from generation 1 through g . The pruning rule is:

$$\text{Prune trial } t \text{ at generation } G^* \iff \tilde{f}(t, G^*) \leq T^* \quad (1)$$

where G^* and T^* are derived from the initial dataset as described above. In the present study, $G^* = 3$ and $T^* = 0.140$.

C. Phase 2: Pruning Rule Validation and Generalization

To validate the derived pruning rule, a new set of trials was initiated with the pruner deployed. Hyperparameter configurations were again sampled randomly using Optuna. For each trial, the median best fitness was monitored, and if it did not exceed the threshold T^* at generation G^* , the trial was terminated.

To evaluate the pruner’s accuracy and assess the rate of false negatives (i.e., promising trials that were incorrectly pruned), 90 randomly selected pruned trials were resumed and allowed to run to completion (generation 17). This counterfactual analysis produced a validation confusion matrix with precision, recall, and F_1 on unseen data, testing whether the pruning rule generalizes.

IV. EXPERIMENTAL RESULTS

A. Justifying the Success Threshold via Sensitivity Analysis

We defined a successful trial as one reaching the 80th percentile of final fitness scores. To validate that this choice was not arbitrary, we conducted a sensitivity analysis to compare it against more permissive (50th, 75th) and stricter (90th) thresholds. For each of these four definitions, we derived the optimal pruning rule by finding the generation (G^*) and fitness threshold (T^*) according to our methodology. Each of these four rules was then evaluated on the independent validation set.

The results, summarized in Table I, reveal the trade-offs between computational savings and the risk of incorrectly pruning a high-potential trial.

TABLE I
SENSITIVITY ANALYSIS FOR THE SUCCESS THRESHOLD PERCENTILE. EACH RULE WAS DERIVED ON THE INITIAL DATASET AND EVALUATED ON THE VALIDATION SET.

Success Def.	Rule (G^*, T^*)	Recall	Precision	F_1 -Score	Savings (%)
50 th	(13, 0.145)	0.52	1.00	0.68	11.4
75 th	(4, 0.143)	0.86	0.97	0.91	38.2
80 th	(3, 0.140)	0.90	0.84	0.87	41.6
90 th	(3, 0.140)	0.94	0.70	0.80	41.6

The 50th percentile was judged ineffective, waiting too long to prune ($G^* = 13$) and still achieving very poor recall. The

90th percentile rule, while preserving almost all elite trials (recall = 0.94), suffered from low precision, allowing too many unsuccessful trials to pass, resulting in a lower F_1 score.

The primary comparison is between the 75th and 80th percentile rules. Although the 75th percentile rule achieved the highest F_1 score (0.91), the 80th percentile rule presented a better overall profile for our objectives. Specifically, the 80th percentile rule:

- 1) **Maximized Recall:** It achieved a recall of 0.90, higher than the 75th percentile's 0.86. For a pruning mechanism, minimizing the risk of discarding valuable "slow starter" trials (False Negatives) is essential, making higher recall a critical feature.
- 2) **Yielded Greater Savings:** As a direct result of pruning earlier, it achieved a computational cost reduction of 41.6%, exceeding the savings of 38.2% from the 75th percentile rule.

The 80th percentile thus gives the best trade-off between safety (high recall) and efficiency for our purposes.

B. Phase 1: Derivation of the Pruning Rule

The initial phase of the experiment aimed to answer two key questions: At which generation can we reliably distinguish between successful and unsuccessful trials? And what is the optimal fitness threshold to use for pruning at that generation?

1) *Identifying the Optimal Pruning Generation G^* :* To determine the earliest point of divergence, the fitness trajectories of trials labeled "Successful" (top 20%) and "Unsuccessful" (bottom 80%) were analyzed. Figure 1 plots the mean of the median best fitness for both classes across the 17 generations.

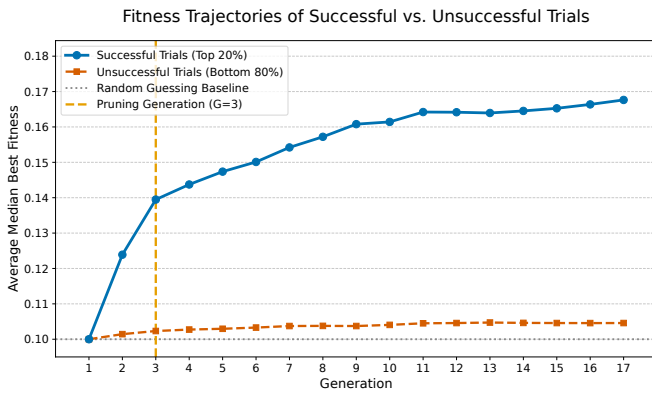


Fig. 1. Fitness trajectories of "Successful" vs. "Unsuccessful" trials. The plot shows the mean of the median best fitness for trials labeled as "Successful" ($n=19$) and "Unsuccessful" ($n=71$).

As illustrated in Figure 1, a clear separation emerges in the early generations. Following the methodology described in Section III-B2, we identified the generation at which successful trials reliably surpassed the 0.1 fitness baseline. This analysis yielded an optimal pruning point of $G^* = 3$. Section IV-F validates this choice statistically.

2) *Determining the Optimal Pruning Threshold T^* :* With the pruning generation fixed at $G^* = 3$, the optimal fitness threshold, T^* , was determined by treating the problem as a binary classification task. For every unique median fitness value observed at generation 3, which served as a candidate threshold, each trial was classified based on its early fitness. This prediction was then compared against the trial's true label.

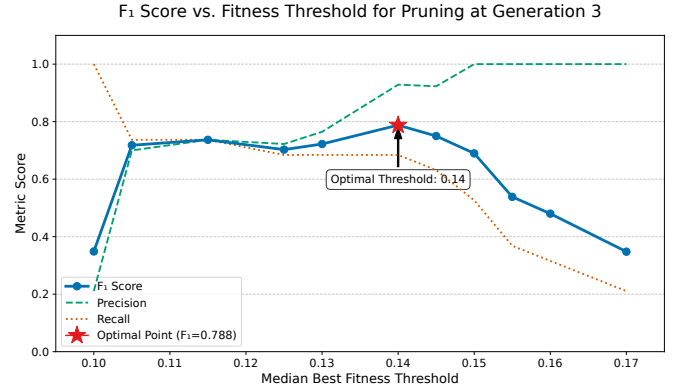


Fig. 2. F_1 score for pruning thresholds at generation $G^* = 3$. The plot shows the F_1 score, precision, and recall for different median best fitness thresholds. The threshold that maximizes the F_1 score represents the optimal trade-off between identifying unsuccessful trials and retaining successful ones.

The analysis, shown in Figure 2, identified an optimal threshold of $T^* = 0.140$. This threshold yielded a maximal F_1 score of 0.788 on the derivation dataset. The $F_1 = 0.788$ reported here is computed on the derivation set (90 trials) where "Successful" is defined as the top 20% final fitness. This setup both (i) yields a strongly imbalanced label distribution (19 Successful vs. 71 Unsuccessful) and (ii) introduces threshold-induced label noise near the 80th-percentile cutoff, since trials with very similar final fitness can fall on opposite sides of the boundary. The derivation F_1 is a conservative measure of separability at generation G^* , rather than an estimate under a prevalence-faithful class prior. This value distinguishes trials demonstrating genuine learning from those stuck at the 0.1 baseline. *The final derived rule was therefore: Prune any trial where the median best fitness at generation 3 is less than or equal to 0.140.*

C. Phase 2: Validation and Generalization

The second phase tested the derived pruning rule on a new, independent set of trials to assess its real-world performance and generalization. The pruner was actively deployed, terminating trials that met the pruning condition.

To quantify the pruner's accuracy, 90 randomly selected pruned trials were resumed to completion to observe their true final fitness. A further 90 trials that ran to completion were selected to form a validation dataset of 180 trials. This allowed for the construction of a confusion matrix based on the pruner's decisions versus the actual outcomes, as shown in Table II.

The matrix reveals 83 True Negatives and 75 True Positives. The primary errors were 8 False Negatives (successful trials

TABLE II
CONFUSION MATRIX FOR THE PRUNER’S PERFORMANCE ON THE
VALIDATION SET. ROWS REPRESENT THE ACTUAL FINAL OUTCOME,
WHILE COLUMNS REPRESENT THE PRUNER’S PREDICTION.

Actual Outcome	Predicted Outcome	
	Successful	Unsuccessful (Pruned)
Successful	75	8
Unsuccessful	14	83

incorrectly pruned) and 14 False Positives (unsuccessful trials that passed).

To quantitatively summarize this performance, a suite of standard metrics was calculated from the confusion matrix, as detailed in Table III.

TABLE III
KEY PERFORMANCE METRICS CALCULATED FROM THE VALIDATION
CONFUSION MATRIX.

Performance Metric	Value
Precision	0.843
Recall (Sensitivity)	0.904
Specificity	0.856
F_1 Score	0.872
Accuracy	0.878
Matthews Correl. Coeff. (MCC)	0.757

The F_1 score of 0.872 and MCC of 0.757 show that the rule classifies well on unseen data. The pruner kept over 90% of truly successful trials (recall 0.904) and correctly rejected most failing ones (specificity 0.856). Of the trials it let through, 84.3% were successful (precision 0.843).

The validation F_1 (0.872) exceeds the derivation F_1 (0.788). This should not be read as an improvement after tuning; the pruning rule is fixed ($G^* = 3$, $T^* = 0.140$). Instead, the difference is expected because the validation confusion matrix is computed on a stratified evaluation set (90 pruned trials resumed + 90 trials that were not pruned and ran to completion), which changes the class prior (Table II) and can make precision/recall (and therefore F_1) differ from the derivation setting.

These validation results confirm that the empirically derived pruning rule generalizes to unseen trials from the same search space.

D. Computational Efficiency

Beyond classification accuracy, the pruning mechanism aims to reduce computational cost. We quantified this by comparing total generations evaluated in our validation set against the number required without pruning.

A trial terminated by the pruner runs for only 3 generations, whereas completion requires 17 generations. Based on Table II, pruning reduced total cost from 3,060 generations (180 trials $\times$ 17 gens) to 1,786 generations ((91 pruned $\times$ 3) + (89 completed $\times$ 17)), achieving a 41.6% reduction while retaining over 90% of successful trials.

Comparison with Hyperband. To assess the performance of our custom pruner, we conducted a parallel experiment using a standard pruner from the Optuna framework. We ran a TPE search combined with the HyperbandPruner (HbP), configured with a `min_resource=2`, `max_resource=17`, and a `reduction_factor=2`. This configuration creates four successive “rungs” where trials are evaluated and potentially pruned.

The performance of our data-driven pruner was compared with this Hyperband baseline across several key metrics, summarized in Table IV.

TABLE IV
COMPARATIVE PERFORMANCE BETWEEN OUR CUSTOM PRUNER, THE
STANDARD HYPERBAND PRUNER, AND THE UNPRUNED TPE BASELINE.

Performance Metric	Our Method	Hyperband	Baseline
Best Final Fitness Found	0.225	0.250	0.275
Avg. Top 5% Fitness	0.206	0.195	0.241
Total Cost (Gens)	10,656	29,574	40,260
Trials to 0.25	Not Reached	569	359

Peak fitness and efficiency trade off against each other. Hyperband found a slightly higher peak fitness (0.250 vs. 0.225), but our pruner produced better configurations on average: the top 5% of our trials averaged 0.206 vs. Hyperband’s 0.195. Hyperband’s broader search occasionally hits a higher maximum, but our method concentrates resources on the productive part of the search space.

The cost difference is large: 10,656 total generations for our pruner vs. 29,574 for Hyperband, a 64% reduction. This efficiency gain comes at the expense of the lower peak fitness noted above, but the domain-specific rule concentrates budget on high-potential trials more effectively than Hyperband’s fixed schedule.

Our method trades peak fitness for search efficiency in this problem domain. We did not match the baseline’s peak fitness, which warrants further investigation into how the pruner behaves with larger trial budgets.

Statistical Comparison. To provide a rigorous quantitative comparison of the overall effectiveness of our pruner, the standard Hyperband pruner, and the unpruned TPE baseline, we analyzed the distributions of all final fitness scores from every completed trial in each study. A one-way Analysis of Variance (ANOVA) was conducted (using all completed trials from the current study combined with the 2,013 trials from prior work [4]), which revealed a statistically significant difference in the mean final fitness achieved across the three methods ($F(2, 3676) = 1090.7$, $p < .001$). At $N = 3,679$, the Central Limit Theorem ensures robustness of the ANOVA to non-normality [14]. A confirmatory Kruskal–Wallis test on the surviving archived data subset ($N = 2,695$) confirmed this result ($H(2) = 421.5$, $p < .001$, $\epsilon^2 = 0.156$), verifying robustness to distributional assumptions without requiring normality. The agreement between parametric (ANOVA) and non-parametric (Kruskal–Wallis) omnibus tests validates the

subsequent Tukey HSD post-hoc comparisons. For the paired t -tests (Table VII), $N = 90$ per group similarly ensures CLT robustness [14].

Tukey’s HSD post-hoc test (Table V) shows that our pruner significantly outperforms Hyperband in mean population fitness, with a large effect size (Cohen’s $d = 1.884$).

No significant difference appeared between our pruner and the unpruned TPE baseline ($p = 0.069$). The pruner finds equivalent solutions at 41.6% lower cost.

TABLE V
TUKEY’S HSD POST-HOC TEST RESULTS COMPARING THE THREE METHODS.

Comparison	Mean Diff	p -value	Sig.	Cohen’s d
Our vs. HbP	0.052	< .001	Yes	1.884
Baseline vs. HbP	0.058	< .001	Yes	1.555
Our vs. Baseline	-0.006	0.069	No	-

Equivalence Testing. While the ANOVA showed no significant difference, this is not strong evidence of equivalence. To make a more rigorous claim, we performed a Two One-Sided Tests (TOST) for equivalence. We defined a practical equivalence margin (Δ) of $\pm 2\%$ absolute fitness (0.02). Given that peak fitness in our experiments is approximately 0.275, this margin corresponds to roughly four additional correctly classified images per 200-image batch, a practically insignificant difference.

Table VI shows the TOST result is significant ($p < .001$), formally rejecting non-equivalence. The 41.6% cost reduction does not degrade solution quality.

TABLE VI
RESULTS OF THE TOST EQUIVALENCE TEST COMPARING OUR METHOD TO THE UNPRUNED BASELINE.

Equivalence Test Metric	Value
Equivalence Margin (Δ)	± 0.020
Observed Mean Difference	-0.0059
TOST p -value	< .001
Conclusion	Statistically Equivalent

E. Analysis of Pruning Errors

Eight successful trials were incorrectly killed (False Negatives). Figure 3 plots their full fitness trajectories.

These trials are “slow starters”: they eventually reach high fitness, but their median best fitness at generation 3 sat below the 0.140 threshold. An aggressive early cutoff inevitably loses a few of these late-blooming runs. The rule accepts this risk because the 41.6% cost savings outweigh the loss of 8 out of 83 successful trials.

False Positives. The 14 false positives (trials that passed early but ultimately failed) result from batch-sampling variance and the relative success definition. These errors reduce computational savings rather than solution quality, as reflected in the precision of 0.843.

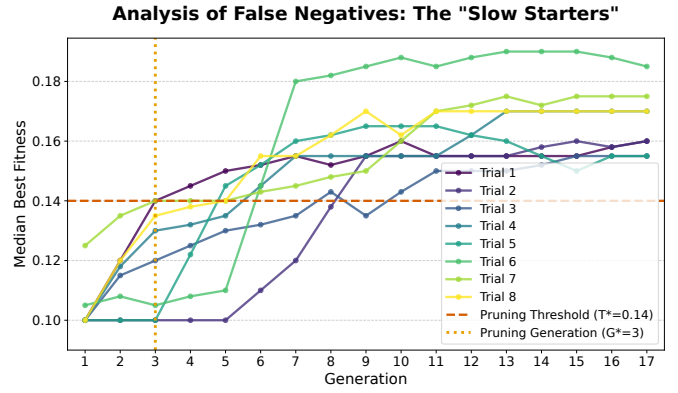


Fig. 3. Fitness trajectories of the 8 successful trials that were incorrectly pruned (False Negatives). These trials failed to surpass the fitness threshold of 0.140 (red dashed line) by the pruning generation $G^* = 3$ (vertical line).

F. Robustness Validation

Metric Selection. Because the 80th percentile of *last best fitness* remained at baseline 0.1 (random guessing), preventing meaningful differentiation, we adopted *median best fitness*. Paired t -tests confirmed no significant difference between metrics ($p > 0.05$, Cohen’s $d < 0.2$; Table VII). At $N = 90$ per group, the t -test is robust to non-normality by the Central Limit Theorem [14].

TABLE VII
RESULTS OF PAIRED-SAMPLES t -TESTS COMPARING FINAL PERFORMANCE METRICS (*Last Median Best* VS. *Last Best*) WITHIN EACH EXPERIMENTAL GROUP.

Experimental Group	t -statistic	p -value	Cohen’s d
Random Search (N=90)	-0.88	0.378	-0.13
Random Unpruned (N=90)	-0.65	0.519	-0.10
Random Pruned (N=90)	-1.00	0.321	-0.15

Validation of $G^* = 3$. To ensure the selected pruning generation of $G^* = 3$ was truly optimal and not merely an artifact of the dataset, we statistically validated its performance against adjacent generations. For each candidate generation ($G \in \{2, 3, 4\}$), we used the pruning threshold derived from our initial, large-scale analysis ($T^* = 0.122$ for $G = 2$, $T^* = 0.140$ for $G = 3$, and $T^* = 0.143$ for $G = 4$).

We then evaluated each of these three fixed rules on our independent, 180-trial validation set. To assess statistical significance, we applied two complementary tests: McNemar’s exact test for paired classifier comparison and a paired bootstrap test (1,000 resamples) for the F_1 -score difference.

Table VIII shows that $G^* = 3$ is significantly better than $G^* = 2$. McNemar’s test confirms a significant difference ($p = 0.022$), with $G^* = 3$ correctly classifying 11 trials that $G^* = 2$ misclassified, while only 2 trials showed the reverse. The paired bootstrap test agrees: the 95% CI for the F_1 difference ($G^* = 3 - G^* = 2$) is $[0.012-0.098]$, excluding zero ($p = 0.003$). Waiting one extra generation produces a more reliable decision.

$G^* = 3$ and $G^* = 4$ do not differ significantly (McNemar’s $p = 0.375$), but $G^* = 3$ intervenes one generation sooner. It is the earliest generation that achieves top classification performance.

TABLE VIII
 F_1 -SCORES AND 95% CONFIDENCE INTERVALS FOR THE PAPER’S PRUNING RULES, EVALUATED ON THE CURATED 180-TRIAL VALIDATION SET. CONFIDENCE INTERVALS WERE CALCULATED VIA BOOTSTRAPPING (N=1,000 RESAMPLES).

Pruning Gen. (G^*)	F_1 -Score	95% Confidence Interval
2	0.819	[0.754–0.878]
3	0.872	[0.817–0.924]
4	0.869	[0.812–0.917]
<i>Paired difference ($G^* = 3 - G^* = 2$): 0.053 [0.012–0.098]</i>		

G. Generalizability Test: Simulation on a Converged Search Space

To test boundary conditions, we simulated our rule on the 824 late-stage trials from prior work [4] that contained per-generation data. This tests whether an early-stage pruner remains effective on a converged, pre-filtered search space.

Table IX summarizes the outcome. The pruner still separates fit from unfit trials ($t(822)=15.9$, $p < .001$), with 73% simulated savings. But the aggressive threshold is too strict for this elite population: recall dropped to 31.1%, and the single best trial would have been killed, reducing peak fitness from 0.275 to 0.245 (Figure 4).

Figure 5 shows the fitness trajectories of these false negatives, the “slow starters” characteristic of converged search spaces. The rule is specialized for random initial populations, motivating adaptive pruners that adjust strictness as search converges.

TABLE IX
 RESULTS OF A COUNTERFACTUAL SIMULATION APPLYING OUR $G^*=3$ PRUNING RULE TO THE FINAL 824 TRIALS OF A PRIOR DATASET.

Late-Stage Simulation Metric	Value
Simulated Computational Savings	73.2 %
F_1 -Score on Late-Stage Trials	0.402
Recall on Top 20% Late-Stage Trials	31.1 %
Mean Fitness of Kept Trials	0.1870
Mean Fitness of Pruned Trials	0.1500
Statistical Separation (p -value)	$< .001$

V. DISCUSSION

The pruning rule, derived from the fitness dynamics of a random initial population, achieved $F_1 = 0.872$ and 41.6% computational savings on the validation set. Against Hyperband, it cut cost by 64.0% while producing higher mean fitness (Cohen’s $d = 1.884$), though Hyperband found a slightly higher peak. Domain-specific pruning outperforms general-purpose schedules when the algorithm’s failure mode is predictable from early trajectories.

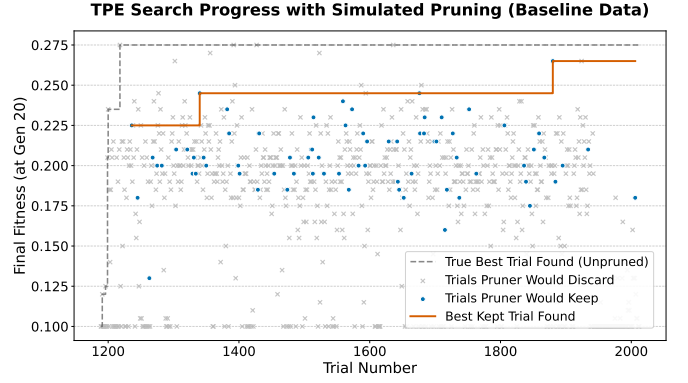


Fig. 4. TPE search progress with our pruner simulated on the prior dataset. Gray crosses mark trials the pruner would have dismissed; blue dots mark trials it would have kept. The dashed gray line shows the true best fitness found over all trials (unpruned baseline), while the solid orange line shows the best fitness achieved using only kept trials.

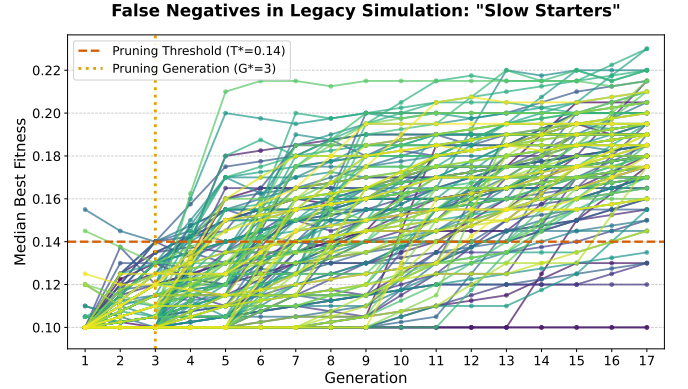


Fig. 5. Fitness trajectories of the “slow starter” trials incorrectly pruned during the baseline simulation (False Negatives).

Schedule-based pruners (SHA, Hyperband) apply fixed elimination ratios at predetermined resource budgets regardless of domain. Our approach instead derives its stopping criterion from the fitness dynamics of the specific algorithm-task pair. T^* and G^* encode a concrete fact about ES-HyperNEAT on MNIST: trials either show learning by generation 3 or never do, and no generic schedule can capture this. The same domain specificity that yields the 64% efficiency advantage over Hyperband also explains the limitation on converged populations: the rule is calibrated for initial random exploration, not late-stage refinement.

The binary learn-or-don’t behavior that makes early stopping possible follows from ES-HyperNEAT’s indirect encoding. The CPPN must produce weight and connectivity patterns that route information from input nodes through the quadtree-generated substrate to output nodes. At generation 0, 93% of our 180 validation trials sit at the random-guessing baseline of 0.1, and 29% never leave it across all 17 generations. Trials still at baseline at generation 3 remain stuck for the full run 87% of the time: small NEAT mutations (adding a node, toggling a connection) rarely bridge the gap between

a substrate with no working signal path and one that propagates input information to outputs. Successful trials show an immediate jump at generation 1 (mean fitness rises from 0.10 to 0.14) and then plateau: mean fitness changes by less than 0.005 between generation 1 and generation 16. The CPPN-to-substrate mapping produces a fitness surface with a large flat region at random guessing and a separate region of viable configurations, and whether a trial lands in one or the other is largely determined at initialization.

A. Limitations and Future Directions

The counterfactual simulation on late-stage trials (recall 31.1%) shows that a rule calibrated for diverse populations is too strict once the search converges and “slow starters” become more common, even though statistical separability holds ($p < .001$).

Adaptive pruning that starts aggressive and relaxes as the population improves would address this [15]. Possible directions include median-based rules that compare intermediate results against completed trials, learning-curve forecasting [9], [16], and hybrid methods like BOHB and DEHB that reallocate budget to late bloomers [17], [18].

The optimal pruning parameters likely depend on task-specific dynamics, so transferability is uncertain. Future work should test on other benchmarks and explore metalearning frameworks that derive problem-specific stopping rules.

The mechanistic account in Section V rests on aggregate trajectory statistics rather than direct landscape measurement. Characterizing the ES-HyperNEAT fitness surface more directly, for example by measuring the basin of attraction at the baseline plateau or the genetic distance between stuck and escaped CPPN initializations, would strengthen the landscape claim.

VI. CONCLUSION

We derived a data-driven early-stopping rule for ES-HyperNEAT that achieves $F_1 = 0.872$ and cuts computational cost by 41.6% without degrading fitness quality. Simulation on converged search spaces exposed the boundary conditions of static rules and points toward adaptive mechanisms that relax the threshold as the population improves. The specific thresholds ($G^* = 3$, $T^* = 0.140$) are tied to ES-HyperNEAT on MNIST, but the methodology (framing pruning as classification on early fitness dynamics) applies to any evolutionary algorithm whose failures are diagnosable from early trajectory statistics.

ACKNOWLEDGMENT

Use of Generative AI: In accordance with IEEE policy, we disclose that Claude [19] (Anthropic) was used to assist with creating figure visualizations and revising drafts to improve grammar, spelling, and consistency of writing style. All AI-generated content was reviewed, verified, and edited by the authors, who take full responsibility for the final work.

REFERENCES

- [1] K. O. Stanley and R. Miikkulainen, “Evolving neural networks through augmenting topologies,” *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.
- [2] S. Risi and K. O. Stanley, “An enhanced hypercube-based encoding for evolving the placement, density, and connectivity of neurons,” *Artificial Life*, vol. 18, no. 4, pp. 331–363, 2012.
- [3] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyperparameter optimization,” in *Advances in Neural Information Processing Systems*, vol. 24, 2011.
- [4] R. Claret, M. O’Neill, P. Cotofrei, and K. Stoffel, “Investigating hyperparameter optimization and transferability for ES-HyperNEAT: A TPE approach,” in *Proc. Genetic and Evolutionary Computation Conference Companion (GECCO ’24 Companion)*. New York, NY, USA: ACM, 2024, pp. 1879–1887.
- [5] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, “Taking the human out of the loop: A review of Bayesian optimization,” *Proc. IEEE*, vol. 104, no. 1, pp. 148–175, 2016.
- [6] J. Bergstra and Y. Bengio, “Random search for hyperparameter optimization,” *J. Mach. Learn. Res.*, vol. 13, no. 10, pp. 281–305, 2012.
- [7] K. Jamieson and A. Talwalkar, “Non-stochastic best arm identification and hyperparameter optimization,” in *Artificial Intelligence and Statistics*. PMLR, 2016, pp. 240–248.
- [8] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A novel bandit-based approach to hyperparameter optimization,” *J. Mach. Learn. Res.*, vol. 18, no. 185, pp. 1–52, 2018.
- [9] T. Domhan, J. T. Springenberg, and F. Hutter, “Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves,” in *IJCAI*, vol. 15, 2015, pp. 3460–3468.
- [10] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [11] K. O. Stanley, “Compositional pattern producing networks: A novel abstraction of development,” *Genetic Programming and Evolvable Machines*, vol. 8, no. 2, pp. 131–162, 2007.
- [12] A. Westh and S. K. Munck, “Pureples – Pure Python library for ES-HyperNEAT,” 2017. [Online]. Available: <https://github.com/ukuleleplayer/pureples>
- [13] A. McIntyre, M. Kallada, C. G. Miguel, C. Feher de Silva, and M. L. Netto, “neat-python, version 2.1.0,” doi:10.5281/zenodo.19024753. [Online]. Available: <https://github.com/CodeReclaimers/neat-python>
- [14] S. G. Kwak and J. H. Kim, “Central limit theorem: The cornerstone of modern statistics,” *Korean J. Anesthesiol.*, vol. 70, no. 2, pp. 144–156, 2017.
- [15] F. Mohr and J. N. van Rijn, “Fast and informative model selection using learning curve cross-validation,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 8, pp. 9669–9680, 2023.
- [16] K. Swersky, J. Snoek, and R. P. Adams, “Freeze-thaw Bayesian optimization,” 2014, arXiv:1406.3896.
- [17] S. Falkner, A. Klein, and F. Hutter, “BOHB: Robust and efficient hyperparameter optimization at scale,” in *Int. Conf. Machine Learning*. PMLR, 2018, pp. 1437–1446.
- [18] N. Awad, N. Mallik, and F. Hutter, “DEHB: Evolutionary hyperband for scalable, robust and efficient hyperparameter optimization,” in *Proc. 30th Int. Joint Conf. Artificial Intelligence (IJCAI-21)*, 2021, pp. 2147–2153.
- [19] Anthropic, “Claude,” 2026. [Online]. Available: <https://www.anthropic.com/claude>